RESEARCH PAPER

ON

PLACEMENT ELIGIBILITY CHECKER

Submitted in partial fulfilment of the requirement for the award of Degree of Bachelor of Technology in Computer Science & Technology

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Nandani Kumari (24EARCS100)

Komal Agarwal (24EARCS077)

Muskan Singh (24EARCS099)

Kashish (24EARCS073)

Nency Jain (24EARCS102)

Under the supervision of

PROJECT GUIDE

Er. Ram Babu Buri

(Assistant Professor)

PROJECT COORDINATOR

Dr. Vishal Shrivastava

Professor

Session: 2025-2026

Department of Information Technology

ARYA COLLEGE OF ENGINEERING & I.T.,

SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

ABSTRACT

Campus placement is an important activity for colleges, students, and recruiting organizations. During placement drives, student profiles must be checked against company-specific eligibility conditions such as minimum CGPA, percentage, active or uncleared backlogs, eligible academic branches, attendance, and required skills. When these checks are carried out manually using spreadsheets or separate records, the process becomes repetitive, time-consuming, and inconsistent. This research presents the design of the *Placement Eligibility Checker*, a desktop application intended to support placement officers in managing student and company information and in evaluating eligibility against configurable criteria.

The proposed system uses Java Swing for the graphical user interface, Java classes for validation and eligibility logic, JDBC for database connectivity, and MySQL for persistent storage. Its main functions include administrator authentication, student registration and record management, company management, criteria configuration, eligibility checking, reason-wise results, searching and filtering, and dashboard summaries. A layered architecture separates presentation, business services, and data access in order to improve maintainability, testability, and security.

The paper describes the problem, objectives, related concepts, requirements, architecture, database design, eligibility algorithm, testing approach, limitations, and possible future enhancements. The system is presented as an academic design based on a Software Requirements Specification (SRS); performance measurements and deployment outcomes must be established through implementation and testing rather than assumed. The work highlights how a transparent, rule-based and explainable workflow can reduce repeated manual checking while keeping final placement decisions with authorized human staff.

KEYWORDS

Placement Management
Eligibility Checking
Java Swing
MySQL
JDBC
Student Records
Rule-Based Decision Support
College Recruitment
Desktop Application
Layered Architecture

INTRODUCTION

Campus recruitment connects graduating students with organizations offering internships and employment. Before a student can participate in a particular placement drive, the college or the recruiting company may specify academic and other conditions. These can include a minimum CGPA or percentage, a maximum number of backlogs, eligible branches, attendance thresholds, and skills relevant to a job role. Because every company can define different conditions, a student's eligibility for one drive may not be the same as their

eligibility for another.

In a manual process, placement personnel compare spreadsheets, student forms, and company notices. The same details may need to be checked repeatedly for several companies. This creates avoidable administrative work and can lead to missed conditions, outdated records, inconsistent interpretation, or unclear explanations when a student does not qualify. A centralized application can reduce repeated checking by storing student profiles and company criteria in a structured database and applying the same configured rules each time.

Placement Eligibility Checker is planned as a desktop application for academic use. Java Swing provides forms and result tables, Java service classes validate data and apply eligibility rules, JDBC connects the application to MySQL, and database tables maintain users, students, companies, criteria, and eligibility results. The system is intended to show an overall status, **Eligible** or **Not Eligible**, together with the individual criteria that passed or failed.

The system is designed as decision support for a placement department, not as a replacement for official placement policy. The college must approve the interpretation of CGPA scales, percentage conversion, active backlogs, missing attendance, branch matching, and skill requirements. Clear rules and traceable reasons are important because eligibility outcomes affect access to placement opportunities.

Problem Statement

The central problem is the repeated and potentially inconsistent comparison of student information with changing company-specific placement rules. A manual process requires staff to retrieve student data, read the relevant company requirements, compare every condition, and communicate the result. When student counts and recruiter participation increase, these activities consume substantial staff time. Errors may arise from:

- Data-entry mistakes in student records
- Duplicate or conflicting records for the same student
- Use of an outdated criteria sheet
- Failure to consider one mandatory condition
- Inconsistent interpretation of the same rule by different staff members

A further problem is that a single yes/no result may not explain why a student is ineligible. Without a criterion-wise explanation, students and placement officers have difficulty identifying whether the issue concerns CGPA, percentage, backlogs, branch, attendance, or skills. The proposed system addresses these issues by centralizing records, validating inputs, applying explicit rules, and presenting unmet requirements together with the result.

Aim and Objectives

The aim is to design a maintainable desktop application that helps a college placement department manage student profiles and company requirements and check eligibility in a consistent, explainable manner. The specific objectives are:

1. Maintain student academic and contact information in a MySQL database.

2. Maintain company records and company-specific eligibility criteria.
3. Validate required fields and numerical ranges before records are saved.
4. Compare each student's details with all mandatory criteria for a selected company.
5. Display the final status and the reasons for failed conditions.
6. Provide search, filtering, updating, and deletion functions for authorized users.
7. Show summary counts for students, companies, and eligibility outcomes.
8. Separate interface code, business logic, and database access to support future maintenance.

Scope of the System

The proposed scope includes administrator login, student registration and maintenance, company management, eligibility-criteria management, eligibility evaluation, result history, searching and filtering, and a dashboard with summary counts. Student self-service login is optional and is not assumed to be present in every implementation. The initial system is a desktop application for Windows, Linux, or macOS where a supported Java Development Kit and a MySQL installation are available.

The system focuses on checking student information against defined placement criteria. It should not be presented as a complete recruitment platform unless additional features are actually developed. Job-posting portals, automated résumé ranking, interview scheduling, messaging with recruiters, and integration with external company systems are outside the core scope unless separately approved.

Users of the System

- **Administrator / Placement Officer:** the primary user who manages student records, company profiles, rules, eligibility checks, and summaries.
- **Student (optional):** may be added to view their own profile, browse company criteria, and check their own eligibility, provided that access control is implemented.
- **Developer / Maintainer:** configures the runtime environment, manages database connectivity, troubleshoots errors, and maintains backups.
- **College:** responsible for supplying accurate records and approved placement rules.

LITERATURE REVIEW

The proposed application combines ideas from database information systems, rule-based decision support, and layered software design. This section reviews the existing ways in which eligibility checking is performed, the technical concepts relevant to the development of the *Placement Eligibility Checker*, and the gaps that justify the proposed design. The review is concept-oriented: it is based on the project SRS and standard software-engineering and database references rather than on any experimental study of a completed system.

Existing Approaches to Eligibility Checking

Manual Spreadsheet-Based Checking

In many colleges, student data is maintained in spreadsheets and company requirements arrive as notices or e-mails. Placement staff filter the sheet manually for every drive. This approach is flexible and requires no special software, and it provides features such as:

- Easy sorting and filtering of student lists
- Quick editing of individual values
- No installation or training requirement

However, a spreadsheet can store records but it does not by itself guarantee that every company rule is checked, that the rules are current, or that rejection reasons are presented in a consistent format. Repeated copying of data between sheets also increases the chance of inconsistent records.

Generic Student Databases and Institutional Software

A generic student database may maintain profiles, academic results, and contact details, and institutional software may offer broader administrative functions. Such systems may not include company-specific eligibility evaluation, configurable criteria for every drive, and a stored result history with reasons. Broader recruitment portals also tend to bring heavy infrastructure, many integrations, and costs that are not suitable for a focused academic implementation.

Technical Background

Relational Database Information Systems

A relational database organizes related information into tables with primary and foreign keys [5][9]. This structure is suitable for keeping student profiles, recruiter information, criteria, and result records linked while reducing unnecessary duplication. Constraints and application-level validation help protect data integrity, for example by:

- Preventing duplicate roll numbers
- Rejecting academic values outside the allowed range
- Preventing results from referencing nonexistent students or companies

Rule-Based Decision Support

Rule-based decision support evaluates a case against explicit conditions. In this project, eligibility is not inferred from a statistical model; it is determined by direct comparison with company-approved thresholds and lists. For example, a minimum CGPA is checked against the student's CGPA, while the maximum allowed backlog count is checked against the recorded backlog count. This approach is understandable and auditable when each condition and its interpretation are documented. It is only as reliable as the data and rules supplied to it.

Layered Software Architecture

Layered architecture separates presentation, business services, and data access [7][8][11]. A

graphical interface collects inputs and displays outcomes, service classes enforce validation and eligibility rules, and data-access objects perform database operations. Keeping raw SQL and core decision rules out of the user-interface layer makes the application easier to test and change. The SRS recommends this structure through Java Swing screens, service classes such as EligibilityService, DAO classes, and a MySQL database.

Java Swing, JDBC and MySQL

Java Swing is a mature toolkit for building desktop interfaces with forms, dialogs, and tables [1]. JDBC provides a standard way for Java programs to connect to relational databases [2], and PreparedStatement allows parameterized queries that reduce the risk of SQL injection [3][12]. MySQL is a widely used open-source relational database and can be accessed from Java through the Connector/J driver [5][6].

Identified Gaps in Existing Approaches

Based on the above review, the following gaps are identified:

1. Spreadsheets do not enforce validation, so out-of-range or duplicate data can enter silently.
2. Company-specific rules are scattered across notices and are easy to apply inconsistently.
3. A bare yes/no outcome does not explain which criterion was not satisfied.
4. Previous eligibility checks are rarely preserved in a structured, searchable form.
5. Generic student databases seldom combine student data, company criteria, and evaluation in one workflow.

Research Gap and Motivation

The project motivation is the gap between fragmented, manual eligibility checking and a centralized workflow that applies approved criteria consistently. The proposed system brings these activities into one workflow: store student data, store company rules, retrieve the selected records, evaluate each criterion, present the result with reasons, and optionally preserve the result for later review. Its emphasis is on transparency, validation, and maintainable design. The research gap should be understood as a project-specific operational need identified by the SRS, not as a claim that no similar placement-management tools exist.

Conclusion of Literature Review

The review shows that although records can be kept in spreadsheets or general databases, there is a clear need for a simple, academic-friendly system that includes:

- Structured storage of student and company data
- Configurable company-specific criteria
- Consistent rule-based evaluation
- Criterion-wise reasons for every negative result
- Validation, security, and maintainable layered design

The *Placement Eligibility Checker* addresses these needs by providing a design that is technically practical and suitable for implementation as an academic project.

PROPOSED SYSTEM

The proposed system, *Placement Eligibility Checker*, is designed to overcome the limitations of manual and fragmented eligibility checking. It offers a reliable, consistent, and explainable platform that lets a placement department store student profiles and company requirements in one place and evaluate eligibility with a single action. The system integrates familiar Java technologies so that it can be built, demonstrated, and maintained within an academic environment.

System Overview

Placement Eligibility Checker is a Java-based desktop application built using:

- Java Swing for the graphical user interface
- Java service classes for validation and eligibility logic
- DAO (data-access) classes for database operations
- JDBC for database connectivity
- MySQL for persistent data storage

The system follows a layered architecture that separates presentation, business services, data access, and the database, ensuring separation of concerns and easy maintenance.

Key Features of the Proposed System

1. Administrator Authentication

Administrative features open only after valid credentials are supplied. Passwords are stored as secure hashes and administrative actions are restricted to authorized users.

2. Student Record Management

An authorized user can create, view, search, update, and delete student records containing:

- Identity and contact details (roll number, name, email, phone)
- Branch
- CGPA and percentage
- Backlog count and attendance
- Skills

3. Company Management

Company records (name, location, job role, website, active status) can be added, viewed, updated, searched, and deleted.

4. Criteria Configuration

For each company the officer can store the eligibility criteria:

- Minimum CGPA and minimum percentage
- Maximum allowed backlogs
- Minimum attendance
- Eligible branches
- Required skills

5. Eligibility Evaluation

The officer selects a student and a company and runs the check. The system compares the student's data against every configured criterion for that company and produces an overall status of Eligible or Not Eligible.

6. Reason-Wise Results

The result screen lists which conditions passed and which failed, so that a negative outcome is always accompanied by explicit reasons such as “CGPA below minimum” or “Too many backlogs”.

7. Search, Filtering and Result History

Results can be filtered by company, student, branch, or eligibility status. When result history is enabled, each outcome is stored with its reasons and a timestamp for later review.

8. Dashboard Summary

A dashboard summarizes total students, companies, eligible outcomes, and not-eligible outcomes, with quick actions to open the main forms.

Functional Requirements

The SRS identifies twelve functional requirements:

- **FR-01:** Administrator authentication before administrative features are opened.
- **FR-02:** Creation of student records with identity, contact, branch, CGPA, percentage, backlog count, attendance, and skills.
- **FR-03:** Student search by student ID, name, or branch.
- **FR-04:** Update or deletion of a selected student after validation and confirmation.
- **FR-05:** Adding, viewing, updating, searching, and deleting company records.
- **FR-06:** Storing company criteria such as minimum CGPA and percentage, allowed backlogs, eligible branches, minimum attendance, and required skills.
- **FR-07:** Comparing student data against every configured criterion.
- **FR-08:** Displaying which conditions passed or failed and producing the final status.
- **FR-09:** Filtering by company, student, branch, or eligibility status.
- **FR-10:** Dashboard summary counts.

- **FR-11:** Understandable database-error messages without exposing passwords or raw secrets.
- **FR-12:** Rejection of missing required fields and values outside approved ranges before saving.

Non-Functional Requirements

- **Security:** password hashing, parameterized database queries, and role checks for administrative actions. Database credentials should be kept outside source control, and real student information should not be included in public repositories or sample data.
- **Usability:** clear labels, consistent controls, readable tables, validation messages, and confirmation dialogs for destructive actions.
- **Reliability:** connection failures and exceptions are handled without silently losing data.
- **Performance:** to be assessed against the expected college dataset; the SRS gives no measured response-time target, so one should be agreed before formal acceptance.
- **Maintainability:** separation of the interface, model classes, services, and DAOs.
- **Data integrity:** primary keys, foreign keys, unique constraints, and validation in both Java and the database where supported.
- **Portability:** documentation of the JDK, MySQL version, JDBC driver, and configuration steps.

Advantages of the Proposed System

- Reduces repetitive manual checking
- Centralizes student and company records
- Applies configured criteria consistently
- Gives understandable reasons for ineligibility
- Provides quicker retrieval of student and company information
- Allows prior checks to be reviewed through result history
- Gives an overview of placement activity through the dashboard

These are expected benefits based on the design; their magnitude must be established through actual use and measurement.

Why This System Is Proposed

This system is proposed because it provides a balance of:

- Technical depth (Java Swing, JDBC, relational design, rule evaluation)
- Real-world application (placement eligibility workflow)
- Academic feasibility (desktop deployment with a standard JDK and MySQL)

- Practical functionality (records, criteria, reasons, filtering, dashboard)

The *Placement Eligibility Checker* fills the gap between unstructured manual spreadsheets and overly complex commercial recruitment suites.

METHODOLOGY

The methodology describes the step-by-step approach used to design, develop, and verify the *Placement Eligibility Checker*. A structured methodology ensures that the project is completed systematically with clarity and precision, and that every requirement can be traced to a design decision and a test.

Development Approach

The project follows a modular, phase-wise development process based on the Software Development Life Cycle (SDLC). The key phases include:

1. Requirement Analysis
2. Data and Architecture Design
3. Interface (Swing) Development
4. Service and Eligibility Logic Development
5. Database Creation and DAO Implementation
6. Validation, Security, and Error Handling
7. Testing and Debugging
8. Deployment and Maintenance

Requirement Analysis

Development begins with requirements analysis based on the SRS. During this phase:

- Functional requirements define what the application must do, including authentication, student and company management, criteria management, eligibility checks, result filtering, and error handling.
- Non-functional requirements address security, usability, reliability, performance, maintainability, data integrity, and portability.
- User roles (Administrator, optional Student, Maintainer) are defined.
- The college's placement policy is collected so that CGPA scale, backlog definition, attendance handling, and skill matching are approved in writing.

System Design

1. Database Design

The proposed database is named *placement_checker* and contains five principal tables:

- **users:** user_id, unique username, password_hash, role, and active status.

- **students:** student_id, unique roll number, name, email, phone, branch, CGPA, percentage, backlogs, attendance, skills, and creation time.
- **companies:** company_id, name, location, job role, website, and active status.
- **eligibility_criteria:** links criteria to a company; stores minimum CGPA, minimum percentage, maximum backlogs, minimum attendance, eligible branches, and required skills.
- **eligibility_results:** links a student and a company; stores the result status, reasons, and timestamp.

Primary keys identify records, and foreign keys connect criteria and results to their parent records. A unique roll number prevents duplicate student registration. The simplified SRS schema allows one active criteria record per company; a more advanced implementation could maintain versioned criteria for historical audit purposes.

Numerical fields use suitable decimal types and are validated at application level. The example SRS schema restricts CGPA to a 0–10 range, percentage to 0–100, backlog count to non-negative values, and attendance to either null or 0–100. The chosen CGPA scale must match the college's official policy. Database CHECK constraints may behave differently across database versions, so Java-side validation remains necessary.

2. Interface Design

Wireframes are prepared for the following screens, using Java Swing components:

- Login
- Dashboard
- Student registration and search
- Company management
- Criteria management
- Eligibility check and result display
- Reports and filtering

Consistent labels, keyboard-friendly controls, adequate contrast, and clear confirmations are used to improve usability.

3. Layered Architecture

- Presentation layer: Swing forms, dialogs, tables, and dashboard
- Service layer: validation, student, company, and eligibility services
- Data-access layer: DAO classes using JDBC
- Database: MySQL tables

Implementation Approach

The implementation stage uses Java Swing for forms, dialogs, tables, and dashboard

components. JDBC connects the Java application to MySQL. Model classes represent users, students, companies, criteria, and results, while DAO classes encapsulate database operations. Service classes validate incoming values and apply business rules before results are stored or displayed.

Core Backend Functions

- Database connectivity via JDBC with reliable closing of resources
- Parameterized PreparedStatement queries for every SQL operation
- Input validation for required fields and numeric ranges
- Password hashing and role-based access checks
- Eligibility evaluation with a list of failed criteria
- Optional storage of the result with a timestamp

Eligibility Algorithm

The eligibility engine evaluates the selected student's data against the criteria associated with the chosen company. The workflow is:

1. Initialize an empty list of failures.
2. If the student's CGPA is below the configured minimum, record "CGPA below minimum".
3. If the percentage is below the threshold, record "Percentage below minimum".
4. If the backlog count exceeds the permitted maximum, record "Too many backlogs".
5. Check whether the student's branch appears in the eligible-branch list.
6. Check whether attendance meets the configured minimum when attendance is required.
7. Check whether the required skills are satisfied.
8. If the failure list is empty, the result is ELIGIBLE; otherwise the result is NOT ELIGIBLE and the failure list is returned for display.

The exact comparison policy must be approved by the placement department, including treatment of missing values, active backlogs, skill matching, and whether rules use AND or OR logic. This approach favors explainability: every negative outcome is associated with one or more explicit conditions. It also makes boundary-value testing possible. A student exactly at a minimum threshold should pass that threshold if the rule is defined as "greater than or equal to," while a backlog count exactly equal to the maximum should pass if the rule is "less than or equal to." These comparison operators must be documented and tested consistently.

Verification Approach

The verification stage uses scenario-based tests derived from the acceptance criteria in the SRS. Types of testing planned:

- Unit-level checks for range validation and eligibility comparisons

- Integration tests for services and DAOs reading and writing MySQL records
- Database testing for insertion, update, retrieval, and connection failure
- Functional testing of login, student, company, criteria, and eligibility modules
- User-interface tests for navigation, input feedback, result display, and confirmation dialogs

Test results must be recorded after the system is implemented; the scenarios described here are planned tests, not claims of completed execution.

Deployment

The system is intended to be deployed as a desktop application on a machine with a supported Java Development Kit and a MySQL installation. A deployment package should include the compiled application or build instructions, the supported Java version, the required MySQL setup, database schema scripts, and a configuration guide. Database credentials and machine-specific settings should be provided through a protected configuration mechanism rather than embedded in public source code.

SYSTEM ARCHITECTURE

The system architecture of the *Placement Eligibility Checker* describes how different components interact to deliver complete end-to-end functionality. The architecture follows a structured, layered design to ensure clarity, maintainability, and ease of testing. Each layer has a specific responsibility and communicates with the next only through defined interfaces.

Architectural Overview

The recommended architecture contains four logical parts:

1. Presentation Layer (Java Swing)
2. Service / Business Layer (Java service classes)
3. Data-Access Layer (DAO classes with JDBC)
4. Database Layer (MySQL)

Presentation Layer

Technologies used: Java Swing (forms, dialogs, tables, dashboard components).

Responsibilities:

- Display the user interface
- Receive user inputs (login, student data, company data, criteria)
- Show eligibility results with pass/fail indicators and a reason panel
- Show dashboard counts and navigation to Students, Companies, Criteria, Eligibility Check, and Reports

Service / Business Layer

Technologies used: Java classes such as AuthenticationService, ValidationService, StudentService, CompanyService, and EligibilityService.

Responsibilities:

- Authenticate users and enforce role checks
- Validate input data before it is saved
- Retrieve student records and company criteria
- Compare student values against each mandatory condition
- Return a status with reasons
- Control whether result history is stored

This layer acts as the “brain” of the entire system. The interface should not contain raw SQL or duplicate the decision rules across multiple screens. If a threshold comparison needs to change, it should be updated in the service layer and tested centrally.

Data-Access Layer

Technologies used: JDBC with parameterized PreparedStatement queries. DAO classes are provided for users, students, companies, criteria (CriteriaDAO), and eligibility results (ResultDAO). Database-specific details remain inside this layer so that connection handling and query changes can be managed in one place, and a schema change can be handled primarily in the data-access and model layers rather than by rewriting every screen.

Database Layer

Technologies used: MySQL database and SQL. Main tables: users, students, companies, eligibility_criteria, and eligibility_results. Each table is linked through primary and foreign keys to maintain data integrity.

System Workflow

Eligibility Check Workflow

1. The placement officer logs in and a session starts.
2. The officer selects a student and a company.
3. The application retrieves the student’s record through StudentService and StudentDAO.
4. The application retrieves the company’s criteria through CompanyService and CriteriaDAO.
5. EligibilityService compares the values against each mandatory condition.
6. A status with reasons is returned.
7. If result history is enabled, ResultDAO stores the outcome and timestamp.

8. The interface displays the final result and the criterion-wise explanation.

Administrator Workflow

1. Administrator logs in.
2. Registers or updates student records.
3. Adds or updates company profiles.
4. Configures eligibility criteria for each company.
5. Runs eligibility checks and applies filters.
6. Reviews dashboard summaries and result history.

Transaction and Error-Handling Strategy

Operations that update related records are designed with transaction boundaries in mind. For example, if saving a placement drive involves storing a drive record and several eligibility criteria, the application should avoid leaving a partially saved configuration if one part of the operation fails. A JDBC transaction can group related changes and commit them only when all required operations succeed; otherwise, the application can roll back the incomplete change and report that the save did not complete.

Error messages shown to users should be understandable and should avoid exposing database credentials, SQL details, or internal stack traces; technical details can be recorded in a protected log. The application should distinguish between invalid user input, a missing record, a duplicate identifier, and a temporary database connection problem, because each situation requires a different response.

Security, Privacy and Error Handling

Student profiles contain personal and academic information and should be accessible only to authorized users. The system should:

- Collect only the fields needed for placement administration
- Hash passwords with a suitable password-hashing approach [13], never store them as plain text
- Use PreparedStatement for database operations to reduce SQL-injection risk [12]
- Enforce authorization checks for administrative functions
- Show messages such as “Unable to connect to database” without exposing credentials
- Require confirmation before deletion and refresh displayed records after changes
- Document and test database backup and restoration procedures

IMPLEMENTATION AND RESULTS

This section explains how the *Placement Eligibility Checker* is intended to be implemented and describes the outcomes expected from each functional module. The design combines interface, business logic, database, and validation components into a unified desktop

application. Because the supplied SRS contains no executed test logs or empirical measurements, the results below are expected outcomes and acceptance criteria, not measured findings. Actual results should be reported only after the tests have been run.

Implementation

Interface Implementation (Java Swing)

The user interface is built using Swing forms, dialogs, tables, and dashboard components. Key implementation points:

- Consistent labels and predictable navigation across all screens
- Related fields grouped together and required fields identified before submission
- Validation feedback placed close to the field that needs attention, explaining how to correct the value
- Result screen that shows the overall decision and retains enough detail for verification

Important screens: Login, Dashboard, Student Registration, Company Management, Criteria Manager, Eligibility Check, and Reports.

Service and Logic Implementation

Service classes expose named methods such as retrieving a student profile, saving company criteria, and evaluating a student for a drive. They return clear outcomes or controlled errors. Examples of key modules:

- **AuthenticationService:** validates credentials, checks the role, and handles sessions.
- **StudentService:** validates and saves student data, rejects duplicates and out-of-range values.
- **CompanyService:** manages company records and their criteria.
- **EligibilityService:** loads the student and criteria, validates that required values are present, evaluates every criterion, and returns the status with reasons.

Database Implementation (MySQL and JDBC)

The relational database stores all data required for system operations. Key database features:

- Foreign keys for relationships between criteria, results, students, and companies
- Unique roll number to prevent duplicate student registration
- Numeric range constraints supported by Java-side validation
- Transactional updates for related changes

Validation Rules

Input validation is a separate responsibility from eligibility evaluation. Validation checks whether values are supplied in a usable format and within a sensible range; evaluation checks whether valid values satisfy the criteria for a particular drive. Keeping these stages separate helps prevent incorrect outcomes.

- **Student identifier:** required and in the institution's chosen format; otherwise the save is rejected with a field-level message.
- **CGPA / percentage:** numeric and within the configured scale; the user is asked for correction and the scale is never guessed.
- **Backlog count:** whole number, zero or greater; negative or non-numeric values are rejected.
- **Branch:** must match an approved branch value or controlled list.
- **Company threshold:** must use the same scale and units as student data; inconsistent criteria are not saved.
- **Required skills:** a consistent vocabulary is used, and the system explains whether skill matching is exact or normalized.

Use-Case Scenarios

- **Scenario A – Checking one student:** the coordinator selects a student and a company. The application loads the stored profile and the current criteria, validates the inputs, and returns a status with criterion-level explanations. If a required field is missing, the application requests correction instead of generating a misleading final result.
- **Scenario B – Reviewing a group of students:** the coordinator chooses a drive and requests a batch check. Every record is evaluated with the same rule set. One incomplete profile must not prevent valid records from being evaluated, and a summary may count eligible, ineligible, and records requiring review.
- **Scenario C – Updating company criteria:** an authorized user changes a threshold after an official company update. New values are validated before saving, and where feasible the previous criteria or a version identifier is retained so that earlier results can be explained.
- **Scenario D – Correcting student information:** a coordinator discovers an incorrectly entered CGPA or backlog count, corrects the record, and runs the check again. The new result is based on the updated data and is distinguished from the old evaluation.

Example Decision Cases

The following examples are illustrative boundary scenarios, not observed experimental results. If a company requires CGPA ≥ 7.0, percentage ≥ 60, backlogs ≤ 0, and a branch included in its approved list:

- A student meeting all four conditions passes these checks.
- A student with CGPA 6.8 fails the CGPA condition even if the other conditions pass.
- A student with one backlog fails when the permitted maximum is zero.
- A student with CGPA 7.4 and zero active backlogs satisfies the CGPA and backlog conditions for a drive that requires CGPA 7.0 and no active backlogs.

The application should list each failed condition so that the outcome can be explained and checked. Before deployment, the college should define the CGPA scale, conversion rules, whether attendance is mandatory, whether skill requirements are exact or category-based, how missing data is handled, and how criteria changes affect stored historical results.

Testing Implementation

Acceptance Test Cases from the SRS

- **TC-01:** valid administrator credentials open the dashboard.
- **TC-02:** incorrect credentials are rejected safely.
- **TC-03:** a valid student is registered and appears in the student table.
- **TC-04:** a CGPA above the allowed scale is blocked by validation.
- **TC-05:** a duplicate roll number is rejected with a useful message.
- **TC-06:** a student who meets every mandatory criterion is shown as ELIGIBLE.
- **TC-07:** a student who fails one mandatory criterion is shown as NOT ELIGIBLE with the reason displayed.
- **TC-08:** an updated student record is persisted and the table refreshes.
- **TC-09:** an unavailable database is handled without an unexpected crash.
- **TC-10:** filtering by company or status shows only matching records.

Additional Test Scenarios

These proposed tests extend the acceptance approach. Expected results should be approved by the placement department and recorded before execution, so that testing checks the software against policy rather than against assumptions.

- **T-01:** student exactly meets every threshold – the approved boundary rule is applied consistently.
- **T-02:** student falls just below one minimum – the failed criterion is identified accurately.
- **T-03:** a required student field is blank – correction is requested or the case is marked for review.
- **T-04:** backlog count is negative or non-numeric – the invalid input is rejected.
- **T-05:** the database is unavailable during retrieval – a safe connection error is shown without false results.
- **T-06:** company criteria are updated – the correct saved criteria are used for a new evaluation.
- **T-07:** two records use the same unique identifier – duplicate data is prevented or clearly handled.
- **T-08:** a batch includes one incomplete profile – the incomplete record is handled without misreporting other results.

For each test, the record should include the test identifier, application build, database version, input data, expected outcome, actual outcome, pass/fail status, date, and tester. Any mismatch should be investigated and retested after correction, which makes it possible to distinguish a software defect from a misunderstanding of the placement policy.

Traceability from Requirements to Verification

- **Student record management:** create, retrieve, update, and validate records; evidence is that the saved record matches the approved input.
- **Company criteria management:** save and reload criteria, including boundary values; evidence is that reloaded values and units are correct.
- **Eligibility evaluation:** compare results against approved examples; evidence is that all criterion outcomes match expected values.
- **Database connectivity:** test normal operation and connection failure; evidence is successful operation and a safe failure message.
- **User feedback:** review common validation and error states; evidence is that messages identify the issue and the next action.
- **Data protection:** review credentials, permissions, and logs; evidence is no unnecessary exposure in screens or logs.

Evaluation Metrics

A complete evaluation requires actual execution evidence. The project team should prepare a test dataset with approved expected outcomes and compare the application's results against them. Useful measures include:

- Number of passed test cases and number of mismatches
- Validation errors detected
- Successful database operations
- Time required to evaluate a representative batch, recorded with the test environment
- Whether the reason displayed matches the criterion that caused the outcome

Correctness should be assessed across normal and boundary cases, such as a student exactly at the minimum CGPA, just below the threshold, zero backlogs, one backlog when none are permitted, a disallowed branch, a missing attendance value, and a company rule with no branch restriction. Usability can be reviewed by asking representative placement staff to complete common tasks without coaching and observing whether labels, validation messages, and the distinction between a final decision and a record requiring review are clear.

Expected Results

After successful implementation and testing, the following results are expected. They are design targets and must be confirmed by measurement.

Result 1: Complete Eligibility Workflow

The officer can register students, add companies and criteria, run an eligibility check, and view the status together with the reasons.

Result 2: Explainable Outcomes

Every negative outcome lists the explicit conditions that failed, so that students and officers can see why a result was produced.

Result 3: Validated and Consistent Data

Missing fields, out-of-range values, and duplicate roll numbers are blocked before saving.

Result 4: Searchable Results

Results and records can be filtered by company, student, branch, or status, and optional history allows earlier checks to be reviewed.

Result 5: Safe Failure Handling

Database failures produce a clear message without exposing credentials, and incomplete operations are rolled back.

Implementation Risks and Mitigation

- **Unclear placement policy:** different staff may interpret the same rule differently. Mitigation: obtain written approval for each criterion and document exceptions.
- **Incorrect or outdated student data:** false eligible or ineligible outcomes may result. Mitigation: validate fields and review data before each drive.
- **Inconsistent rule scales:** invalid comparisons between CGPA and percentage. Mitigation: store units and grading scales explicitly.
- **Database connection failure:** records cannot be loaded or saved. Mitigation: show a clear message, preserve entered data where safe, and log technical details.
- **Unauthorized access:** exposure or modification of student information. Mitigation: apply access controls and secure database credentials.
- **Untraceable criteria changes:** past decisions become difficult to explain. Mitigation: record rule versions or change history where feasible.

Risk management should be reviewed during development rather than only at the end. A small prototype can verify the connection between Swing forms, the eligibility service, and the database before all screens are built. Reviewing representative edge cases early is especially useful because the most serious defects may be policy-interpretation errors rather than visible programming errors.

Deployment and Maintenance

Before first use, the administrator should create the database, apply the schema, create authorized accounts, and load only approved initial data. A short verification checklist should confirm that the application starts, connects to MySQL, retrieves a student, saves a permitted

update, evaluates a sample drive, and displays a clear message when a required service is unavailable. The checklist should be repeated after material software or database changes.

Maintenance may include changes to placement policies, database fields, supported Java versions, or report formats. Changes should be tested against existing acceptance cases so that a new feature does not silently alter previous eligibility behavior. A short change log and version label can help staff identify which build was used for a particular drive.

CONCLUSION

The *Placement Eligibility Checker* presents a practical design for applying a rule-based desktop application to an administrative process that otherwise depends on repeated manual comparison. By integrating Java Swing, Java service classes, JDBC, and MySQL, the proposed system centralizes student profiles and company requirements and evaluates each student against the configured rules. Its core workflow includes authentication, student and company management, criteria configuration, criterion-wise evaluation, result explanations, filtering, and summary views.

The layered architecture separates presentation, business logic, and database access, while relational constraints and input validation support data integrity. The rule-based approach is intended to provide understandable results by listing the conditions that fail rather than presenting an unexplained decision.

Expected Benefits

- Reduced repetitive checking and quicker retrieval of records
- Consistent application of configured criteria
- More understandable reasons for ineligibility
- Result history and dashboard summaries for review of placement activity

Limitations

- Decisions depend on the accuracy and completeness of stored records and the correctness of company criteria.
- Ambiguous policies, stale data, or inconsistent skill names can lead to inappropriate results.
- The system cannot resolve conflicting policy documents or evaluate a criterion that has no defined scale or operational meaning.
- Skill matching may require a clearly specified method; a simple text comparison can miss synonyms.
- The desktop architecture requires a supported Java runtime and database setup, and may be less convenient for multi-campus use than a centrally hosted system.
- Student self-service, e-mail notifications, résumé uploads, audit logging, automated backups, report export, and a web-based version are future enhancements rather than guaranteed features of the baseline design.

However, the reliability of outcomes depends on approved placement policies, accurate data, secure implementation, and systematic testing. The design should therefore be treated as a foundation for an academic implementation, with actual performance and test outcomes reported only after they have been measured. The system should support human review where data is incomplete or a policy exception is required, and the placement department should retain authority over final decisions.

FUTURE SCOPE

While the *Placement Eligibility Checker* is presented as a proposed design with a clear baseline scope, it has significant potential for expansion. With advancements in web technologies and automation tools, the system can be enhanced into a more complete placement-support platform. The following enhancements can improve usability, security, and scalability, and each should be prioritized according to demonstrated need, privacy impact, and implementation cost.

Student Accounts

Separate student accounts with profile-limited permissions would allow students to:

- View their own records
- Browse company criteria
- Check their own eligibility without gaining administrative access

Report Export

Exporting eligibility results to CSV or PDF could support reporting and communication. Reports should identify the company or drive, the rule version used, the date of evaluation, and the relevant student identifier.

E-mail Notifications and Announcements

E-mail notifications and placement-drive announcements could inform students about relevant opportunities, subject to college approval and privacy safeguards.

Résumé Upload and Skill Profiles

Résumé upload and skill-profile management could improve the quality of student information, although skill matching would need a clearly defined normalization policy.

Audit Logs and Rule Versioning

- Record who changed student records or company criteria and when
- Keep versioned criteria so that earlier results can be explained
- Distinguish old and new evaluations instead of overwriting history

Automated Backups and Recovery

Automated database backups and tested recovery procedures would improve operational

resilience.

Web-Based Migration

If remote access becomes a requirement, the desktop application could be migrated to a web-based architecture such as Spring Boot or another approved technology stack, allowing multi-campus use through a centrally hosted system.

Role-Based Permissions and Institutional Integration

Further development could explore role-based permissions, secure report generation, and carefully governed integration with institutional student-information systems.

Further Research Directions

- **Time and correctness study:** compare the time required for routine eligibility checks and the correctness of outcomes against a manual process, using the same approved student records and company criteria, and report the number of records tested and the conditions.
- **Policy explainability:** provide a concise explanation for each criterion and identify the rule version used.
- **Coordinator feedback:** collect structured feedback about clarity, ease of use, error recovery, and confidence in the displayed explanations, with institutional approval and without exposing identifiable student records.

These steps would help evaluate the tool as an administrative support system while keeping policy decisions with authorized human staff.

Suggested Roadmap for Future Work

To keep the evolution of the system controlled, the enhancements above can be introduced in stages, each followed by regression testing against the existing acceptance cases so that a new feature does not silently change earlier eligibility behavior:

1. **Stage 1 – Stabilize the baseline:** complete the core records, criteria, eligibility check, and recorded test log.
2. **Stage 2 – Strengthen governance:** add audit logs, criteria versioning, and tested database backups.
3. **Stage 3 – Improve reporting:** add CSV or PDF export and richer filtering of results.
4. **Stage 4 – Controlled student access:** add student accounts with profile-limited permissions.
5. **Stage 5 – Communication:** add e-mail notifications and announcements after privacy review.
6. **Stage 6 – Web migration:** evaluate a web-based architecture if remote or multi-campus access is required.

REFERENCES

[1] Oracle – Java Swing

Java Platform documentation, The Java Tutorials.

Available at: https://docs.oracle.com/javase/tutorial/uiswing/

[2] Oracle – JDBC Basics

The Java Tutorials.

Available at: https://docs.oracle.com/javase/tutorial/jdbc/basics/

[3] Oracle – Prepared Statements

The Java Tutorials, JDBC Basics.

Available at: https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html

[4] Oracle – Java Platform, Standard Edition Documentation

Java SE and JDK official documentation.

Available at: https://docs.oracle.com/en/java/

[5] Oracle – MySQL 8.4 Reference Manual

Relational database management, SQL, and constraints.

Available at: https://dev.mysql.com/doc/refman/8.4/en/

[6] Oracle – MySQL Connector/J Developer Guide

JDBC driver for MySQL.

Available at: https://dev.mysql.com/doc/connector-j/en/

[7] Sommerville, I.

Software Engineering, 10th ed. Pearson, 2015.

[8] Pressman, R. S., and Maxim, B. R.

Software Engineering: A Practitioner's Approach, 9th ed. McGraw-Hill, 2019.

[9] Elmasri, R., and Navathe, S. B.

Fundamentals of Database Systems, 7th ed. Pearson, 2016.

[10] Project Source Document

Placement Eligibility Checker – Software Requirements Specification, version 1.0, supplied for this paper.

[11] Fowler, M.

Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.

[12] OWASP – SQL Injection Prevention Cheat Sheet

Guidance on parameterized queries.

Available at:

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

[13] OWASP – Password Storage Cheat Sheet

Guidance on secure password hashing.

Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html